

MATO SEIHEI NO SLAVE — BATTLE ARENA 00

Documento de Consignas, Requerimientos y Guía de Construcción Académica

MATERIA
ADOO / PDS 2026

TECNOLOGÍA
Java 17 & Swing / Web

ARQUITECTURA
Patrones GoF & GRASP

ENTREGA
Informe Final Integrador

Propósito del Documento

El presente documento reúne la totalidad de los enunciados, requerimientos funcionales, especificaciones técnicas y trazabilidad metodológica aplicados para la creación del simulador de combate táctico basado en el universo de *Mato Seihei no Slave*. Sirve como fuente primaria de origen para justificar las decisiones de diseño y la evolución del código.

1. Enunciado General y Alcance del Sistema

El objetivo principal del Trabajo Práctico Integrador consiste en diseñar e implementar un sistema de combate por turnos orientado a objetos en Java 17, complementado con una interfaz rica y una arquitectura extensible que aplique patrones de diseño reconocidos (GoF), principios GRASP, SOLID y patrones de persistencia/arquitectura.

Alcance Funcional Mandatorio

- Equipos de 1 a 3 Waifus:** Selección dinámica con validación estricta de no duplicidad entre bandos (Aliado / Oponente).
- Mecánicas de Turno:** Selección explícita de atacante, objetivo, estrategia táctica y habilidades especiales.
- Estrategias Cambiantes:** Implementación de posturas Agresiva, Equilibrada y Defensiva.
- Turno de IA Enemiga:** Selección automática priorizando mayor ATK en ataque y menor HP en defensa.

Requerimientos Técnicos

- Persistencia de Estado:** Guardado y reanudación de partidas mediante archivos de propiedades / LocalStorage.
- Desacoplamiento UI:** Uso explícito del patrón Observer para registros de combate y sistema de logros.
- Entorno Portable:** Generación de binarios Java Swing y versión Web ejecutable offline.
- Límite de Roster:** Exactamente 8 capitanas oficiales en juego.

2. Roster Oficial de Capitanas y Estadísticas Base

El dominio debe modelar estrictamente las siguientes 8 capitanas del Cuerpo Anti-Demonios, manteniendo coherencia entre el backend Java, la interfaz Swing y la versión Web:

#	Capitana	Elemento	ATK	DEF	HP Máx	Habilidad Especial
1	Konomi Tatara	WIND	82	45	185	BEAST POWER (+35 ATK)
2	Bell Tsukiyono	LIGHT	78	48	190	CANOPUS (+25 ATK, +20 DEF)
3	Yakumo Ezo	WIND	88	52	195	NIGHT STORM (+40 ATK)
4	Tenka Izumo	LIGHT	84	58	205	AME-NO-MITORI (+30 ATK, +15 DEF)
5	Kyouka Uzen	SHADOW	86	55	210	SLAVE (+45 ATK)
6	Varvara Pilipenko	SHADOW	80	70	220	PANTHEON (+40 DEF)
7	Fubuki Azuma	WATER	79	64	215	SUNSET (Curación hasta +55 HP)

3. Requerimientos de Arquitectura y Patrones GoF (12 Patrones)

El sistema debe incorporar y demostrar en la práctica al menos 12 patrones de diseño formalmente estructurados:

Patrones Creacionales y Estructurales

- Factory Method** WaifuFactory y SpecialAbilityFactory para centralizar la instanciación de personajes y habilidades.
- Composite** SquadComponent, Squad y SquadMember para tratar individual y colectivamente a los equipos.
- Decorator** PowerUpDecorator y ArmorDecorator para añadir modificadores dinámicos sobre las Waifus.
- Adapter** AnimeApiAdapter para integrar datos provenientes de modelos externos simulados.
- Facade** AnimeGameFacade como único punto de entrada simplificado a la lógica de negocio.
- Singleton** DatabaseConnection para asegurar una única instancia de conexión simulada.

Patrones de Comportamiento y Persistencia

- Strategy** CombatStrategy con variantes Agresiva, Equilibrada y Defensiva para el cálculo de daño.
- State** WaifuState (HealthyState, InjuredState, KnockedOutState) ajustando conducta según el HP.
- Observer** Notificación de eventos de batalla desvinculada hacia la UI, Log en Consola y Sistema de Logros.
- DAO & Repository** WaifuDAO y WaifuRepository para el aislamiento de datos.
- Table Data Gateway** WaifuTableDataGateway representando acceso directo a tabla en memoria.

4. Reglas de Negocio del Sistema de Combate

Fórmulas de Cálculo de Daño por Estrategia

- Estrategia Agresiva:** $\text{Daño} = \text{ATK atacante} + 20 - (\text{DEF objetivo} / 3)$
- Estrategia Equilibrada:** $\text{Daño} = \text{ATK atacante} + (\text{DEF atacante} / 5) - (\text{DEF objetivo} / 2)$
- Estrategia Defensiva:** $\text{Daño} = \text{ATK atacante} - (\text{DEF objetivo} / 2)$

En todos los casos, el daño infligido nunca podrá ser menor a 1 unidad.

MECÁNICA DE ESTADOS DE SALUD (STATE PATTERN)

- HealthyState (Saludable):** HP > 50% del máximo. Recibe daño de forma estándar (100%).
- InjuredState (Herida):** HP ≤ 50% del máximo y > 0. La vulnerabilidad aumenta; el daño recibido se incrementa en ~15%.
- KnockedOutState (KO):** HP = 0. La unidad queda incapacitada para atacar o recibir acciones.

5. Casos de Uso del Sistema (CU-01 a CU-07)

ID	Caso de Uso	Flujo Principal / Resultado Esperado
CU-01	Iniciar Combate	Seleccionar de 1 a 3 personajes por bando, validar que no haya repetidos entre equipos y reiniciar estados.
CU-02	Atacar con Waifu	Seleccionar atacante, objetivo y estrategia. Aplicar fórmula de daño, actualizar HP/Estado y emitir evento a Observers.
CU-03	Usar Habilidad Especial	Verificar disponibilidad (uso único por combate), ejecutar boost o curación y marcar como consumida.
CU-04	Turno de la IA Enemiga	Selección automática: mayor ATK en atacante enemiga y menor HP en objetivo aliada. Ejecutar ataque.

6. Requerimientos de Calidad, GRASP y SOLID

La construcción del código debe alinearse estrictamente con los principios de diseño orientado a objetos:

Principios GRASP Aplicados

- **Information Expert:** Waifu gestiona sus propios atributos de vida, estado y poder.
- **Controller:** WaifuController recibe eventos de UI y coordina con la Facade.
- **Low Coupling & High Cohesion:** Paquetes aislados funcionalmente; desacoplamiento vía interfaces.
- **Pure Fabrication:** Repositorios, DAOs y Repositorios de Imágenes como clases de servicio.
- **Indirection:** Uso de Adapters y Facades para evitar dependencias directas con subsistemas.

Principios SOLID Demostrados

- **SRP:** Dominio, vistas, multimedia y persistencia separados en paquetes independientes.
- **OCP:** Incorporación de nuevas estrategias o habilidades sin alterar el código existente.
- **LSP:** Sustitución transparente de SquadMember y Squad bajo SquadComponent.
- **ISP:** Interfaces pequeñas y focalizadas (e.g., Observer, CombatStrategy).
- **DIP:** El dominio depende de abstracciones de persistencia y estrategia, no de clases concretas.

7. Guía Pasos de Construcción del Trabajo Práctico

Para reproducir o defender la metodología de desarrollo ante la cátedra, el proyecto se estructuró en cinco fases evolutivas:

1. **Fase 1 — Modelado de Dominio Base:** Creación de la entidad Waifu, jerarquía de estados (WaifuState) y estructuración de equipos mediante el patrón Composite (SquadComponent).
2. **Fase 2 — Encapsulamiento de Algoritmos y Patrones:** Implementación del patrón Strategy para algoritmos de daño, Decorators para PowerUps, y Factories para la instanciación de las 8 capitanas y habilidades especiales.
3. **Fase 3 — Infraestructura, Persistencia y Facade:** Creación del Singleton de base de datos simulada, capa DAO, Repository y Table Data Gateway. Encapsulamiento total mediante AnimeGameFacade.
4. **Fase 4 — Interfaces de Usuario (Swing & Web):** Construcción de WaifuGameFrame en Java Swing y réplica exacta del flujo de batalla en HTML5/JS. Implementación del patrón Observer para actualización de consola, interfaz y logros.
5. **Fase 5 — Refactorización, Verificación e IA:** Identificación de Code Smells (extracción de MediaLauncher e ImageRepository), verificación de compilación en Java 17 y Node.js, y empaquetado portable con recursos limpios.

8. Documentación Académica Obligatoria Incluida

El repositorio final entregado debe contener en la carpeta docs/AD00/ los siguientes archivos explícitos:

- 01_CASOS_DE_USO.md: Detalle completo de actores, precondiciones y flujos alternativos.
- 02_DIAGRAMAS_SECUENCIA.puml: Diagramas PlantUML para ataques, habilidades y turno enemiga.
- 03_GRASP.md & 04_SOLID.md: Justificación técnica punto por punto de la aplicación de principios.
- 05_CODE_SMELLS_REFACTORIZACIONES.md: Reporte de deuda técnica identificada y corregida.
- 06_MVC_OBSERVER.md: Explicación del desacoplamiento de interfaz y flujo de notificaciones.
- 07_IA_APLICADA_AL_DISEÑO_00.md: Bitácora de uso de IA como asistente de diseño e ingeniería de software.
- 08_HABILIDADES_Y_COMBATE.md: Ficha técnica balanceada del roster final.